

PA1 Discussion: Setting up AWS and Dask

DSC 102: Systems for Scalable Analytics (FA26)

Presenters: Qiyu Li

Adapted from: Aniruddha Das, Vraj Shah, Arun Kumar, Sai Sree Harsha

Logistics

[Quick Start: AWS link](#)

Ask questions on Piazza, Help your groupmates with setup

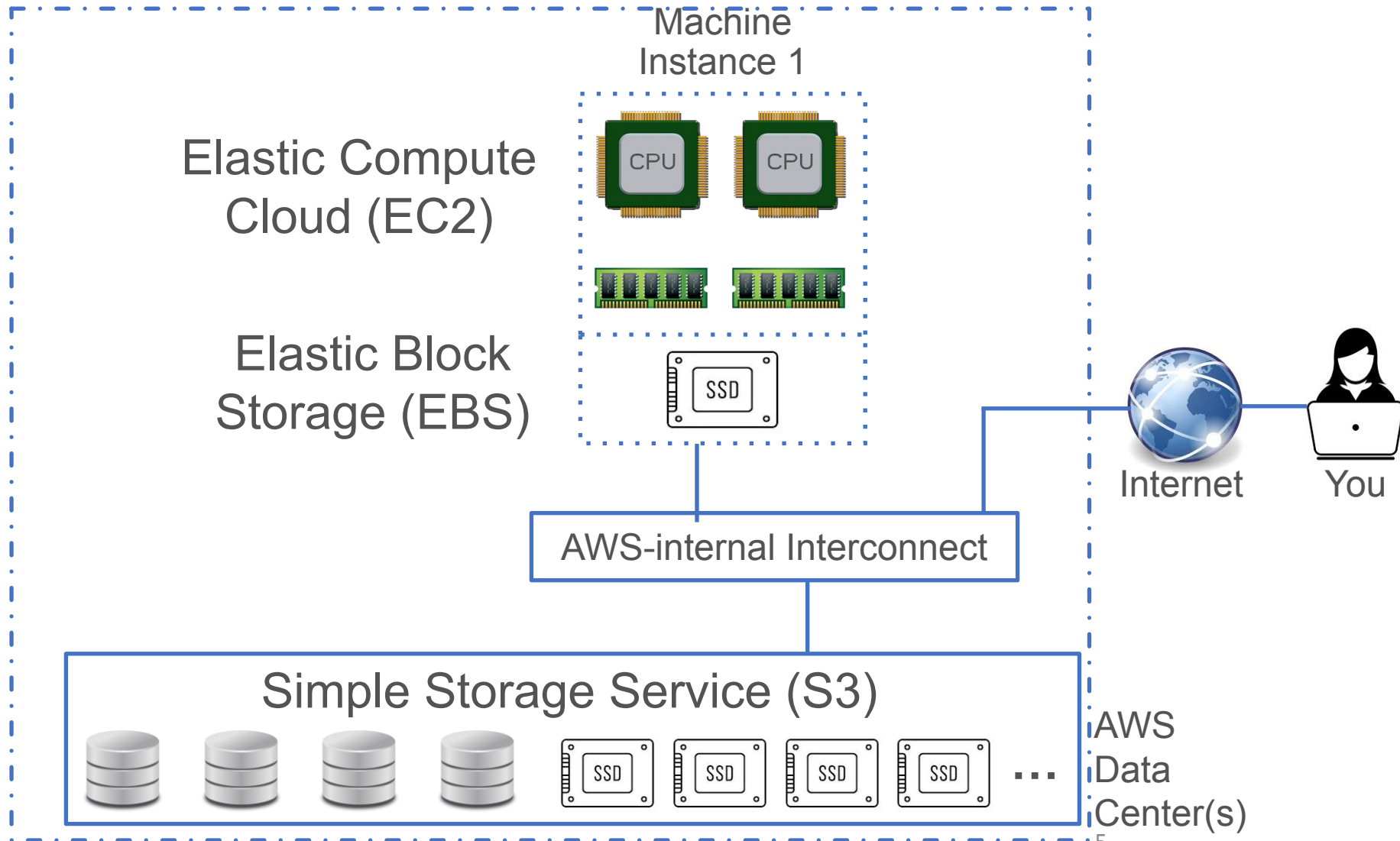
PA1 Assignment Learning Outcomes

1. Interact with the AWS GUI and commonly used resources
2. Experience managing multiple terminals and port forwarding
3. Understand the need for cloud computing by running a dask program on a data file that is too big to fit in a single-node machine

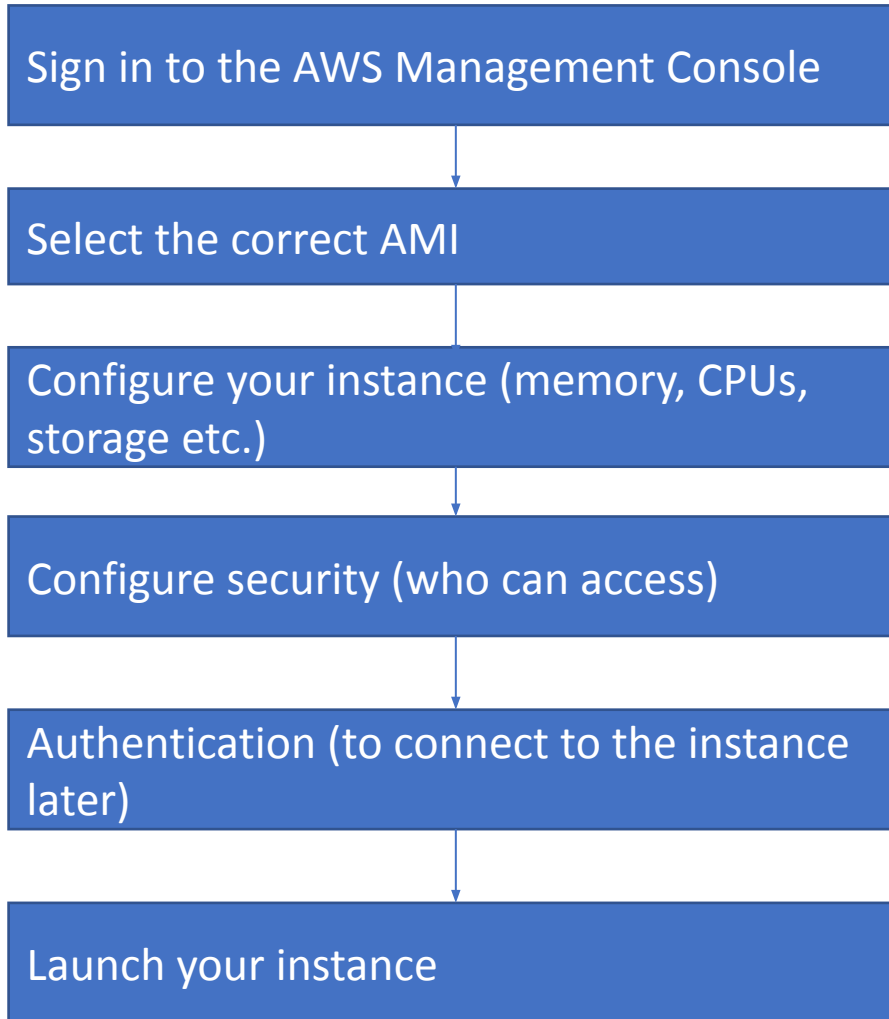
PA1 Discussion Learning Outcomes

1. Identify which AWS resource is associated with each component of PA1
2. Access AWS resources
 - a. Navigate AWS to launch an EC2 Instance
 - b. Access the instance from your local terminal
 - c. Read data from an S3 bucket
 - d. Open your programming environment
3. Run dask on data files that are too big to fit in a single-node machine

AWS Services



Launching an EC2 instance



Connecting to an EC2 instance (SSH)



Dask: Overview



Parallel computing framework that scales existing Python ecosystems



```
np.zeros((10000,10000,10000)) -> OOM!
```

```
dask.array.zeros((10000,10000,10000))) -> SUCCESS!
```

Breaks up work into tasks and executes them in task parallel manner *lazily*

Dask provides APIs to create task graphs, also called *Directed Acyclic Graph* (DAG)
Directed Acyclic Graph (DSC 40) - nodes are linked by one-way connections, no cycles

Dask also provides a scheduler that runs this DAG by assigning tasks to workers

Under the Hood: Task Graphs

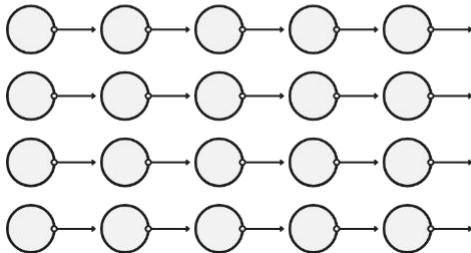
Common Approach to Task Scheduling for Parallel Execution:

“Tasks are collected into a DAG and organized with dependencies and relationships to say how they should run” -Dask Documentation

- DAG Structure:
 - nodes: medium-sized tasks
 - edges: if one task depends on data produced by another
- Task scheduler executes graph and parallelizes (dask uses full task sch.)

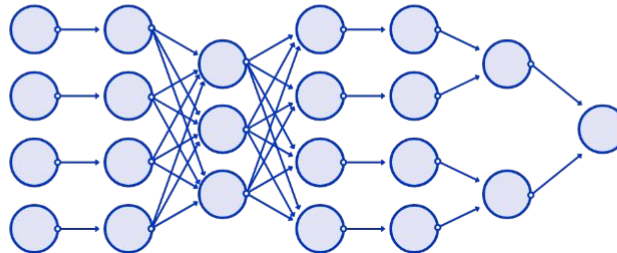
Embarrassingly Parallel

Hadoop/Spark/Dask/Airflow/Prefect



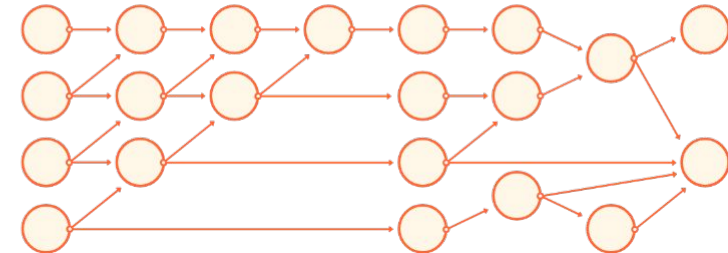
MapReduce

Hadoop/Spark/Dask



Full Task Scheduling

Dask/Airflow/Prefect



Dask: APIs

Low-level APIs:

- Dask Delayed (Parallel lazy objects)
- Dask Futures (Parallel eager objects)

High-level APIs:

- Dask Array (Parallel NumPy)
- Dask DataFrame (Parallel Pandas)
- Dask Bag (Parallel Dictionary)
- Dask ML (Parallel Scikit-Learn)

DataFrame APIs enough for this assignment, feel free to check out other APIs if needed

Dask: DataFrame API

Consists of multiple smaller Pandas DFs

Smaller DFs can be present on disk of a single or multiple machines

Operation on Dask DF triggers operations on smaller Pandas DFs

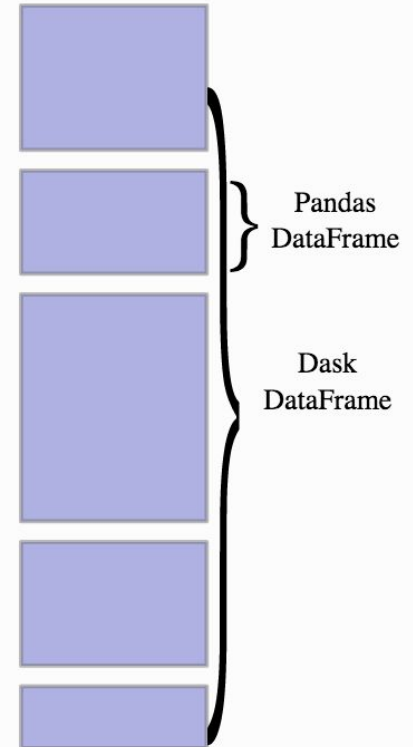
Similar APIs as Pandas DF APIs

```
import pandas as pd
df = pd.read_csv('filename.csv')
```

```
import dask.dataframe as dd
df = dd.read_csv('filename.csv')
```

Use `.compute()` method to trigger computation

```
df.column.mean().compute()
```



Assignment: Dataset Description

Column name	Example
reviewerID	A32DT10X9WS4D0
asin	B003VX9DJM
reviewerName	Slade
helpful	[4, 5]
reviewText	this was a gift for my friend who loves touch lamps.
overall	1
summary	broken piece
ter	1397174400
reviewTime	04 11, 2014

Reviews table

Assignment Tasks: PA 1

Column name	Example
reviewerID	A32DT10X9WS4D0
asin	B003VX9DJM
reviewerName	Slade
helpful	[4, 5]
reviewText	this was a gift for my friend who loves touch lamps.
overall	1
summary	broken piece
ter	1397174400
reviewTime	04 11, 2014

Reviews table

(1) Create a new *users* dataframe using only the *reviews* table with the following schema:

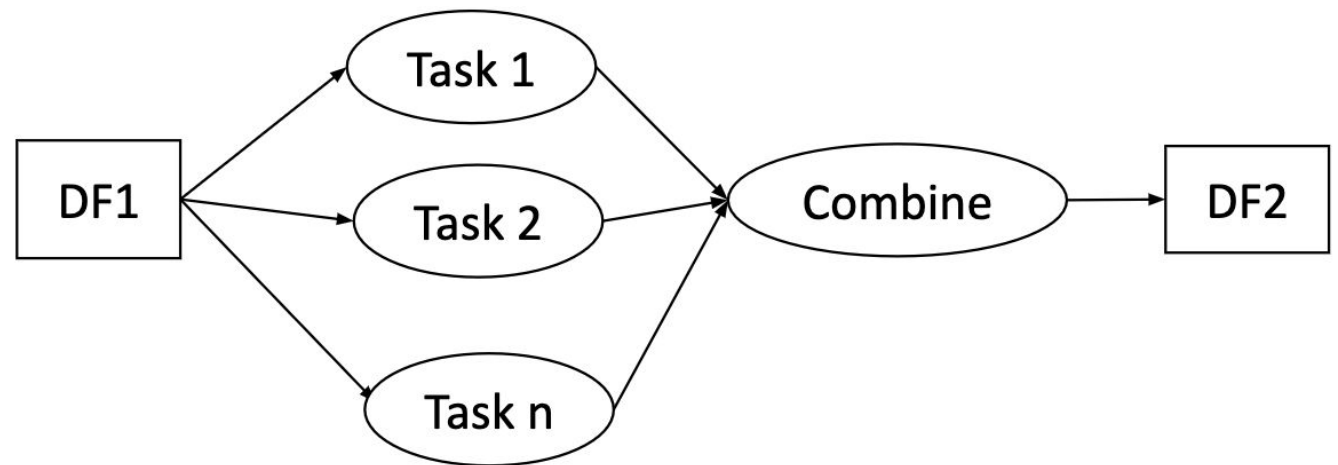
Column name	Column example
reviewerID (PRIMARY KEY)	A32DT10X9WS4D0
number_products Rated	20
avg_ratings	3.5
reviewing_since	2011
helpful_votes	15
total_votes	34

Suggested Approach

Break up the “task” (creating a new users dataframe) into multiple sub-tasks (creating columns of the users dataframe)

Specify operations using Dask DataFrame APIs, which generates the task graph

Given the task graph, Dask scheduler will take care of computing them in a task-parallel manner



Grading Outline

Accuracy(80)

- 5 columns
- 16 points per column
- Error tolerance 1%

Runtime(20)

Absolute 4 node runtimes (20)	Points
Under 5 mins	20
Between 5 mins to 10 mins	12
Between 10 mins to 30 mins	8
Anything above 30 mins	0

We tolerate 1% error margin, but solutions should match exactly on element-wise basis to ours

Sanity Check: Compare your outputs to OutputSchema_PA1 before submitting

See Document on Canvas for More Details

Best Practices for PA1

- Use private GitHub repo if possible for handling code and logs.
- Terminate the AWS instance every time after usage; launch again & read from S3 again next time to save budget.
- Since the development set is large, work on a much smaller subset first. Move to the full dataset later.
- Some helpful dask APIs
 - groupby
 - map_partitions
 - apply
- While performing groupby aggregations on large #groups (millions or more), use **split_out** to split output into multiple partitions to avoid worker error. (see [this](#))

Other Helpful Links

- [Managing Memory](#)
- [Managing Computation](#)
- [Dask Dataframe Best Practices](#)

Demo

Any Questions?